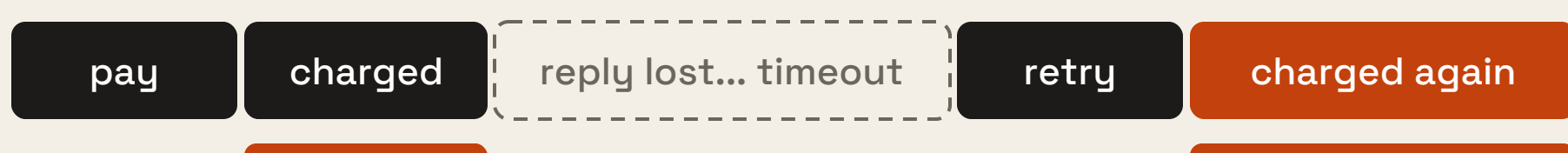


a 2-minute read, promise

Why one retry charges them twice

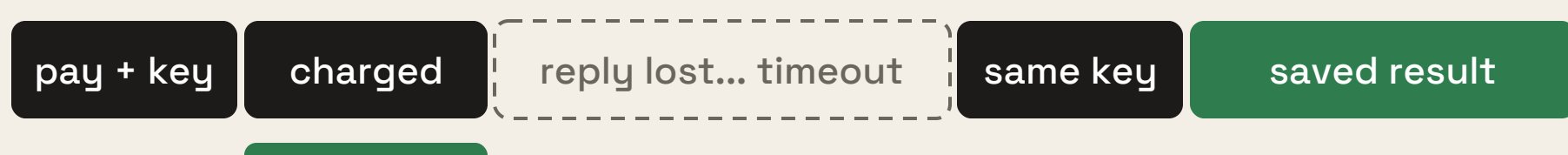
Your app sends a payment. The server charges the card. Then the reply gets lost on the way back. The app sees a timeout, tries again, and the server happily charges again.

Without a key



one click, two charges

With an idempotency key



one click, one charge. The retry just gets the old answer.

Like a token at a food counter. Show the same token twice and you get your order again, not a second plate. The key is that token for your payment.

1 timeout + 1 retry = 2 charges. Unless the server knows it's the same click.

Is this you?

- ☐ Your payment call retries on timeout.
- ☐ You don't send any key with it (or a new one each time).
- ☐ A user once said "I got charged twice" and you shrugged.

Even one tick? Keep reading.

The sneaky version

This passes code review. There's a key. It just changes every try:

```
for attempt in range(3):
    key = uuid4()      # new key, every retry
    pay(order, key)
```

Make the key once when the user taps Pay, save it with the order, reuse it.

Fix it

- 1 Create the key once per click, not per retry.**
Store it on the order row so even an app restart reuses it.
- 2 Put a unique index on the key.**
If two retries land together, only one gets through.
- 3 Same key but the amount changed? Reject it.**
Don't quietly return the old result. Stripe errors here too.
- 4 Keep keys longer than your longest retry.**
Stripe's v1 API can clean them up after 24 hours.

Useful? I break down one backend mistake like this every day.

@gouravkargwal